Steps to use mutex locks. Check program lock.c

1. Declare mutex lock as a global variable.

```
pthread_mutex_t lock;
```

2. Initialize it (may be inside main() function).

```
pthread_mutex_init(&lock,NULL);
```

3. Inside the threads, just aquire lock and release lock.

```
pthread_mutex_lock(&lock);

//Critical section

pthread_mutex_unlock(&lock);
```

POSIX Unnamed Semaphores steps

1. Include Headers:

```
#include <pthread.h>    // pthread_create, pthread_join
#include <semaphore.h>  // sem_init, sem_wait, sem_post, sem_destroy
#include <unistd.h>     // sleep (optional, for demonstration)
```

2. Declare Shared Resource and Semaphore:

- Declare the shared resource that will be protected by the semaphore.
- Declare a sem_t variable to represent the unnamed semaphore.

```
int shared_data = 0; // The shared resource

sem_t my_semaphore; // The semaphore variable
```

3. Initialize the Semaphore:

- Use sem_init() to initialize the semaphore.
- The key argument is pshared: set it to 0 for thread synchronization within the same process.
- Set the initial value to 1 for a mutex (binary semaphore) or higher if you want to allow multiple threads concurrent access up to the value.

```
    int main() {
  int result = sem_init(&my_semaphore, 0, 1); // 0 means inter-thread, 1 is the initial value

  if (result != 0) {
     perror("sem_init failed");
     exit(EXIT_FAILURE);
  }

  // ... rest of the code (thread creation, etc.) ...
}
```

4. Create Threads:

- Use pthread_create() to create the threads that will access the shared resource. Each thread will have a function that uses the semaphore to synchronize access.

```
    void *thread_function(void *arg); // Forward declaration

int main() {
   // ... semaphore initialization (from step 3) ...

   pthread_t thread1, thread2;

   if (pthread_create(&thread1, NULL, thread_function, NULL) != 0) {
      perror("pthread_create failed");
      exit(EXIT_FAILURE);
   }

   if (pthread_create(&thread2, NULL, thread_function, NULL) != 0) {
      perror("pthread_create failed");
      exit(EXIT_FAILURE);
   }

   if (pthread_join(thread1, NULL) != 0) {
      perror("pthread_join failed");
      exit(EXIT_FAILURE);
   }

   if (pthread_join(thread2, NULL) != 0) {
      perror("pthread_join failed");
      exit(EXIT_FAILURE);
   }

   // ... semaphore destruction (from step 6) ...

   return 0;
}
```

5. Thread Function: Wait, Access, Post:

- Inside the thread function:
    - Call sem_wait() to acquire the semaphore. If the semaphore's value is 0, the thread will block until another thread calls sem_post().
    - Access the shared resource (the *critical section*).
    - Call sem_post() to release the semaphore.

```
  void *thread_function(void *arg) {
    printf("Thread started.\n");

    // Wait for the semaphore
    if (sem_wait(&my_semaphore) == -1) {
        perror("sem_wait failed");
        pthread_exit(NULL); // or exit(EXIT_FAILURE) if main thread
    }

    // Critical section: Access shared resource
    printf("Thread entered critical section.  Current shared_data value: %d\n", shared_data);
    shared_data++; // Increment the shared resource
    printf("Thread exiting critical section.  New shared_data value: %d\n", shared_data);
    sleep(1); // Simulate some work inside the critical section

    // Release the semaphore
    if (sem_post(&my_semaphore) == -1) {
        perror("sem_post failed");
        pthread_exit(NULL);  // or exit(EXIT_FAILURE) if main thread
    }

    printf("Thread finished.\n");
    pthread_exit(NULL);
}
```

6. Destroy the Semaphore:

- After all threads have finished, destroy the semaphore using sem_destroy().

```
    int main() {
  // ... thread creation and joining ...

  if (sem_destroy(&my_semaphore) == -1) {
      perror("sem_destroy failed");
      exit(EXIT_FAILURE);
  }

  return 0;
}
```

Complete Example Code:

```
    #include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

int shared_data = 0;
sem_t my_semaphore;

void *thread_function(void *arg) {
   printf("Thread started.\n");

   if (sem_wait(&my_semaphore) == -1) {
      perror("sem_wait failed");
      pthread_exit(NULL);
   }

   printf("Thread entered critical section.  Current shared_data value: %d\n", shared_data);
   shared_data++;
   printf("Thread exiting critical section.  New shared_data value: %d\n", shared_data);
   sleep(1); // Simulate some work

   if (sem_post(&my_semaphore) == -1) {
      perror("sem_post failed");
      pthread_exit(NULL);
   }

   printf("Thread finished.\n");
   pthread_exit(NULL);
}

int main() {
   int result = sem_init(&my_semaphore, 0, 1);

   if (result != 0) {
      perror("sem_init failed");
      exit(EXIT_FAILURE);
   }

   pthread_t thread1, thread2;

   if (pthread_create(&thread1, NULL, thread_function, NULL) != 0) {
      perror("pthread_create failed");
      exit(EXIT_FAILURE);
   }

   if (pthread_create(&thread2, NULL, thread_function, NULL) != 0) {
      perror("pthread_create failed");
      exit(EXIT_FAILURE);
   }
```

```
    if (pthread_join(thread1, NULL) != 0) {
        perror("pthread_join failed");
        exit(EXIT_FAILURE);
    }

    if (pthread_join(thread2, NULL) != 0) {
        perror("pthread_join failed");
        exit(EXIT_FAILURE);
    }

    if (sem_destroy(&my_semaphore) == -1) {
        perror("sem_destroy failed");
        exit(EXIT_FAILURE);
    }

    printf("Final shared_data value: %d\n", shared_data);  //Should be 2

    return 0;
}
```

Compilation and Execution:

Compile using: gcc -pthread your_file.c -o your_program
Run: ./your_program

Explanation:

1. **sem_init(&my_semaphore, 0, 1):** Initializes the semaphore. 0 means it's for threads within the same process. 1 means it's initialized in the "unlocked" state (one thread can immediately acquire it).
2. **sem_wait(&my_semaphore):** Decrements the semaphore. If it was already 0, the thread blocks until another thread calls sem_post. This enforces mutual exclusion.
3. **sem_post(&my_semaphore):** Increments the semaphore. If any threads are blocked in sem_wait, one of them will be unblocked.
4. **sem_destroy(&my_semaphore):** Releases the resources associated with the semaphore.

This example ensures that only one thread at a time can access and modify the shared_data variable, preventing race conditions and data corruption. The sleep(1) is added to give you a good demonstration. Without it, the first thread might enter and exit the critical section so fast that the second thread has no chance to enter.